

Mini Transport Tycoon - Project Wiki

Team Members

Team Name: Team 404

- Khayrullo** - Core Game Logic & Economy
 - Game state management (GameModel, GameState, TimeManager)
 - Economic system (income, costs, bankruptcy detection)
 - Save/load functionality
 - Game speed controls and pause/resume
 - Integration testing and system coordination
- Ulukbek** - Map & Infrastructure
 - Map system and tile management
 - Road construction and removal logic
 - Terrain system (heights, slopes, terraforming)
 - Traffic lights implementation
 - City and industrial facility placement/management
- Ramez** - Vehicles & Routes
 - Vehicle system (purchasing, movement, aging)
 - Route management and pathfinding
 - Stop placement and management
 - Garage system (maintenance, vehicle lifecycle)
 - Goods and passenger transport logic
- Komron** - GUI & Rendering
 - User interface design and implementation
 - Map rendering and visualization
 - Minimap implementation
 - Continuous movement animations
 - User input handling and interaction

Team Size: 4 members

Technology Stack: Java

Short Description of the Game

Mini Transport Tycoon is a simplified transportation-economic simulation game inspired by Transport Tycoon and OpenTTD. The game challenges players to build and manage a road transportation network connecting cities and industrial facilities, transporting goods and passengers to maximize profit.

Core Gameplay

- Map:** 2D grid-based top-down view with pre-placed cities and industrial facilities
- Infrastructure:** Players build roads connecting cities and facilities
- Transportation:** Purchase vehicles (freight trucks and buses) to transport goods and passengers
- Economy:** Manage finances through income from deliveries and costs for infrastructure and vehicles
- Routes:** Define circular routes between stops for vehicles to follow automatically
- Time Management:** Real-time gameplay with adjustable speed (pause, normal, fast, very fast)

Game Objective

Maximize profit by efficiently organizing transportation networks while managing costs. The game ends if the player goes bankrupt (capital becomes negative).

Chosen Subtasks

The base game has a complexity of **1.5 units** (for 4-person teams). To reach the required **5.0 units total**, we have selected the following subtasks totaling **3.5 units**:

Selected Subtasks

- Terrain** (1.0 unit)
 - Variable tile heights with visual representation
 - Roads can only be built between tiles with same height or ± 1 difference
 - Vehicles move slower on uphill slopes
 - Player can terraform terrain (change height) for a cost
- Traffic Lights** (1.0 unit)
 - Player can install traffic lights at intersections (3 or 4-way junctions)

- Configurable green signal duration per direction
 - Vehicles must stop at red lights
 - Visual representation of light states on the map
3. **Garage** (0.5 unit)
- Player can build garages connected to the road network
 - Vehicles can be purchased and maintained in garages
 - Automatic maintenance scheduling based on vehicle age
 - Option to sell old vehicles
4. **Minimap** (0.5 unit)
- Large scrollable game map
 - Navigable minimap for overview and quick navigation
 - Scrollable in both X and Y dimensions
5. **Continuous Movement** (0.5 unit)
- Smooth animation of vehicles between tiles
 - Vehicles logically remain on single tiles but visually move continuously
 - Enhanced visual experience

Total Subtask Complexity: 3.5 units

Base Game Complexity: 1.5 units

Total Project Complexity: 5.0 units ✓

Functional Requirements

Core Game Requirements

FR-1: Map System

- The game must display a 2D grid-based map
- The map must contain pre-placed cities (minimum 3×3 tiles each)
- The map must contain pre-placed industrial facilities (minimum 2×2 tiles each)
- Cities must have an internal, uneditable road network
- The map must be larger than the visible area and scrollable
- Each tile must be visually represented with appropriate graphics

FR-2: Road Construction

- Player must be able to build roads on empty tiles
- Roads cannot be built on tiles occupied by cities or facilities
- Roads can connect to city/facility edges
- Road construction must have a cost
- Roads must be visually distinct on the map
- Roads must support bidirectional vehicle movement

FR-3: Goods and Demand System

- The game must include at least 3 different types of industrial goods
- The game must support passenger transportation
- Industrial facilities must produce specific goods
- Industrial facilities must consume specific goods
- Cities must demand passengers and/or finished products
- Demands must change slowly over time (not instant)
- Goods must be visible at facilities (e.g., cargo icons)

FR-4: Vehicle System

- The game must support road vehicles only
- There must be distinct vehicle types for each cargo category
- There must be buses for passenger transport
- There must be at least 2 different vehicle types per category
- Vehicle types must differ in speed, capacity, and maintenance cost
- Vehicles must be purchasable by the player
- Vehicles must have a purchase cost
- Vehicles must have maintenance costs
- Vehicles must be visually represented on the map

FR-5: Vehicle Movement

- Vehicles can only travel on the road network
- Vehicles must follow assigned routes
- Only one vehicle per direction can occupy a road tile at a time

- Maximum 2 vehicles can be on one tile (one each direction)
- Vehicles must load cargo/passengers when available at stops
- Vehicles must unload cargo/passengers at appropriate destinations
- Vehicles must have visual representation showing their direction

FR-6: Stops and Routes

- Player must be able to place stops along roads
- Stops must occupy one tile
- Stops can be placed near cities and industrial facilities
- Player must be able to create circular routes (e.g., $A \rightarrow B \rightarrow C \rightarrow A$)
- A stop can belong to multiple routes
- Vehicles must automatically repeat their assigned route
- Routes must be visually indicated on the map
- Stops must be visually distinct

FR-7: Economy System

- Player must start with initial capital
- Player must earn income from successfully delivered goods
- Player must earn income from successfully delivered passengers
- Player must pay costs for building roads
- Player must pay costs for purchasing vehicles
- Player must pay costs for vehicle maintenance
- Current capital must be displayed to the player
- If capital becomes negative, the game must end (bankruptcy)

FR-8: Time Management

- The game must run in continuous time
- Player must be able to pause the game
- Player must be able to set speed to normal
- Player must be able to set speed to fast (approximately 2x)
- Player must be able to set speed to very fast (approximately 4x)
- Current game speed must be clearly indicated
- Game time/date must be displayed

Subtask Requirements

FR-9: Terrain System (Subtask)

- Tiles must have variable heights
- Height differences must be visually represented (slope tiles)
- Roads can only be built between tiles with same height or ± 1 difference
- Vehicles must move slower on uphill slopes
- Player must be able to terraform terrain (change height by 1) for a cost
- Absolute height difference between adjacent tiles must be at most 2

FR-10: Traffic Lights System (Subtask)

- Player must be able to install traffic lights at intersections (3 or 4-way)
- Traffic lights must only be placeable on player-built road intersections
- Player must be able to configure green signal duration per direction
- Traffic lights must switch between directions at fixed intervals
- Vehicles must stop at red lights
- Current light color (red/green) must be clearly visible on the map

FR-11: Garage System (Subtask)

- Player must be able to build garages
- Garages must be connected to the road network
- Vehicles must be purchasable in garages
- Vehicles must be maintainable in garages
- Vehicles must automatically return to garage for maintenance at intervals
- Maintenance frequency must increase with vehicle age
- Player must be able to sell old vehicles
- Garage must be visually distinct on the map

FR-12: Minimap System (Subtask)

- The game map must be larger than the displayed area
- The map must be scrollable in both X and Y dimensions

- A minimap must be displayed in the game interface
- The minimap must show the entire game map
- The minimap must show the current viewport
- Player must be able to navigate by clicking on the minimap
- The minimap must be clearly visible and distinguishable

FR-13: Continuous Movement (Subtask)

- Vehicles must move smoothly between tiles (not jumpy)
- Vehicles must be logically on a single tile at any time
- Visual movement must be animated and continuous
- Animation must not affect game logic (collision detection, etc.)

User Interface Requirements

FR-14: Main Game View

- The game must display the map in a 2D top-down view
- All game elements must be visible (cities, facilities, roads, vehicles, stops)
- The view must be scrollable using mouse or keyboard
- Zoom functionality (optional but recommended)

FR-15: Information Display

- Current capital must be displayed
- Game time/date must be displayed
- Current game speed must be displayed
- Selected object information must be displayed when clicked

FR-16: Menu System

- Main menu must be accessible
- Game menu (pause, save, load, settings) must be accessible
- Vehicle purchase menu must be accessible
- Route creation/editing interface must be accessible
- Settings menu must be accessible

FR-17: Input Handling

- Mouse clicks must be handled for map interaction
- Keyboard shortcuts must be available for common actions
- Right-click context menus should be available where appropriate
- Drag operations must be supported for route creation

Data Management Requirements

FR-18: Game State Management

- Game state must be maintained throughout gameplay
- Vehicle positions and states must be tracked
- Cargo/passenger counts must be tracked
- Economic state (capital, costs, income) must be tracked
- Time state must be tracked

Validation Requirements

FR-19: Action Validation

- Invalid road placements must be prevented
- Insufficient funds must prevent purchases
- Invalid route creation must be prevented
- Invalid stop placement must be prevented
- Clear error messages must be shown for invalid actions

Non-Functional Requirements

Performance Requirements

Response Time

- **GUI Responsiveness:** All user interactions (clicks, menu selections) must respond within 100ms
- **Frame Rate:** The game must maintain at least 30 FPS during normal gameplay

- **Frame Rate (Fast Speed):** Must maintain at least 20 FPS at 4x speed with maximum vehicles
- **Map Scrolling:** Smooth scrolling without lag when navigating large maps

Scalability

- **Map Size:** Support maps up to 256×256 tiles minimum
- **Vehicle Capacity:** Support at least 50 vehicles simultaneously without performance degradation
- **Route Capacity:** Support at least 20 routes simultaneously
- **Save File Size:** Save files should not exceed 10MB for typical game states

Resource Usage

- **Memory:** Application should not exceed 512MB RAM for typical gameplay
- **CPU:** Should run smoothly on systems with dual-core processors
- **Disk Space:** Installation should require less than 100MB

Usability Requirements

User Interface

- **Intuitive Controls:** Game should be playable without reading extensive documentation
- **Visual Feedback:** All user actions must provide immediate visual feedback
- **Error Messages:** Clear, user-friendly error messages for invalid actions
- **Tooltips:** Context-sensitive help tooltips for UI elements
- **Keyboard Shortcuts:** Common actions should have keyboard shortcuts

Accessibility

- **Color Blindness:** UI should not rely solely on color to convey information
- **Text Readability:** All text must be readable at standard screen resolutions
- **Icon Clarity:** Icons should be clear and distinguishable

Reliability Requirements

Stability

- **Crash Prevention:** Application should handle errors gracefully without crashing
- **Data Integrity:** Save files must not become corrupted
- **Exception Handling:** All exceptions must be caught and logged appropriately

Data Persistence

- **Save Reliability:** Save functionality must work 100% of the time
- **Load Reliability:** Loaded games must accurately restore game state
- **Auto-save:** Optional auto-save feature to prevent data loss

Maintainability Requirements

Code Quality

- **Modular Design:** Code should be organized into logical modules
- **Documentation:** All public methods and classes must be documented
- **Code Standards:** Follow Java coding conventions
- **Design Patterns:** Use appropriate design patterns (MVC, Observer, etc.)

Portability Requirements

Platform Support

- **Operating System:** Must run on Windows, macOS, and Linux
- **Java Version:** Requires Java 11 or higher
- **Dependencies:** Minimize external dependencies

Technology Stack

- **Programming Language:** Java
- **GUI Framework:** JavaFX (recommended)
- **Build Tool:** Maven or Gradle
- **Version Control:** Git
- **Documentation:** Markdown, PlantUML for diagrams
- **Testing:** JUnit for unit testing

Wiki Subpages

- [Use Case Diagram](#)
- [UI Plan](#)
- [User Stories](#)
- [Class Diagram](#)

Comments